

430.211 Programming Methodology (프로그래밍 방법론)

Taesup Moon

Department of Electrical and Computer Engineering
Seoul National University

Outline

- Intro & Basic course info
- Chapter I: C++ Basics
 - Programming paradigm
 - Comparison between C, C++ and Python
 - Simple C++ program

Outline

- Intro & Basic course info
- Chapter I: C++ Basics
 - Programming paradigm
 - Comparison between C, C++ and Python
 - Simple C++ program

Intro & Basic course info

- Basic survey
 - <https://forms.gle/rqUFsQks36pkJvzQ7>
 - Please do it right now with your phone or laptop!



Intro & Basic course info

- Instructor: **Taesup Moon** (문태섭)

- Rm 301-707, tsmoon@snu.ac.kr
- Office Hour: By appointment



- Brief Intro

- Where I have been to so far
 - SNU EE BS '02
 - Stanford EE MS ('04) / PhD ('08)
 - Yahoo! ('08-'12) / UC Berkeley STAT ('12-'13) / Samsung ('13-'15)
 - DGIST ('15-'17) / SKKU ('17-'21) / SNU ('21.3-**Today**)
- Main research area: ML / AI / Data Science

Intro & Basic course info

- Lab1 TAs

- Yongho Kim (김용호, peterkim98@snu.ac.kr)
- JuhyunPark (박주현, parkjh9229@snu.ac.kr)
- JungYoon Hwang (황정윤, danieljyh@snu.ac.kr)

- Lab2 TAs

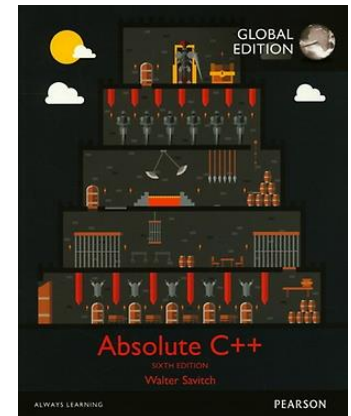
- Heewoong Choi (최희웅, chw0501@snu.ac.kr)
- Sumin Yu (유수민, ysmsoomin@snu.ac.kr)
- Hyunwoong Bae (배현웅, hwbae0326@snu.ac.kr)

- **eTL & 엘리스(elice)** is the main communication channel

- You can also e-mail the instructor and TAs

Lectures and Lab Sessions

- Lectures (**Wed**, 15:30pm-17:20pm) by the Instructor
 - English lectures
 - Textbook: **Absolute C++** (6th or 5th edition)
 - By Walter Savitch and Kenrick Mock
 - English version or 한글 번역본 (by 퍼스트북).
 - Pdf version uploaded to eTL
- Lab 1 (**Fri**, 15:30pm-17:20pm) by the TAs (Head: 김용호)
 - Lecture review, problem solving, HW solution/review
- Lab 2 (**Fri**, 15:30pm-17:20pm) by the TAs (Head: 최희웅)
 - Linux, C++ applications/STL, Git, In-depth Projects,



Grading policy

- Homework (**20%**)
 - Lab1 (**10%**), Lab2 (**10%**)
- Projects (**22%**)
 - Project 1 (**10%**), Project 2 (**12%**)
- Midterm Exam (**22%**)
 - October 22, 3:30pm (tentative)
- Final Exam (**31%**)
 - December 17, 3:30pm (tentative)
- Attendance (**5%**)
- Participation Bonus (**more on this**)

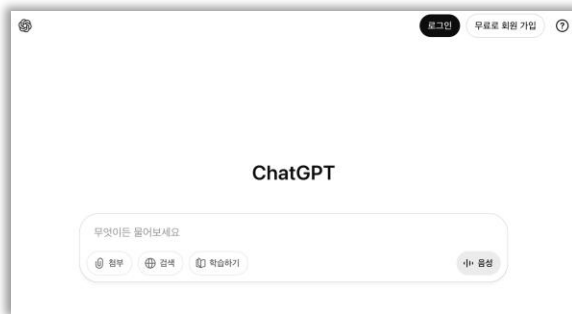
Grading policy - Homework

- HW will be handed out every week from Lab 1/2
 - All HW must be done individually
- Copy & Plagiarism
 - **Strictly** prohibited
 - **Integrity** is more important than a grade!
 - University/academia is a place for ladies and gentlemen.
 - When copying is caught, **both** students who **showed** and **copied** will get **F** grades (Do not make innocent victims!)
 - Discussing together about HW and projects is okay, but you need to code individually!



Grading policy - Homework

- AI tools (ChatGPT, Claude, Gemini, ... etc)
 - Two sides of a sword (beneficial vs. harmful)
 - Actively using ChatGPT for generating answers for the HW is regarded as a **plagiarism!**
 - In fact, it makes us think deeply about the future of programming itself...
 - HWs are for improving your skills / understanding about programming, not for your grade!



Grading policy - Projects

- There will be 2 projects
 - Project 1: **TBD** (week 8)
 - Project 2: **TBD** (week 12)
- Each project will have 3~4 weeks to work on
 - More details will be given in Lab2 sessions
- Projects will be more challenging than past years (due to AI tools)
 - Team of 2
 - Some final questions will be about projects

**DO NOT
COPY**

Grading policy – Delay

- Use cases
 - No delayed submission allowed
 - When d days are late, you will get
$$\max(100 - d \times 20, 0)\%$$
of your score

Grading policy - Attendance

- Your attendance score will be $5 - (n - 1) \times 0.25$
 - n is the number of absence (1 absence has no penalty)
 - Some exceptions on n
 - Valid academic reason (e.g., attending a conference, etc.)
 - Serious matters (e.g., reserve force training, attending a direct family member funeral, etc.)
 - Serious illness
 - Regular dental appointment, mild stomachache are not acceptable
 - In all above cases, you need to **contact the lecturer** (both for the Lecture and the Labs) **via e-mail 24 hours prior to the class**
 - Attendance will be checked via random sampling or Google Forms questionnaire

Grading policy - Participation

- Active participation is highly encouraged
 - After each lecture, a student can submit a Google Form with your name & question to confirm
 - Depending on the question quality, a bonus point will be used for final grading (Particularly, when the grade is on the boundary)
 - Point formula: $\frac{(\text{\# of weeks he/she asked a good question})}{14} \times 10$
 - You can ask questions **anytime** – class is a safe environments!
 - In fact, this is a bit childish, but please see this:
<https://www.youtube.com/watch?v=O6IWUpL2QRU>

Prerequisites and Emphasis

- Prerequisites
 - Basic computer skills
 - Knowledge/experience of high-level programming languages, e.g., Python
- Emphasis
 - Concepts and programming practices of the Object-Oriented Programming (OOP) paradigm
 - Experience of C++ programming skills

Schedule for Lecture/Lab I

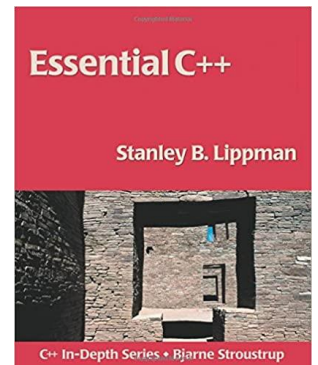
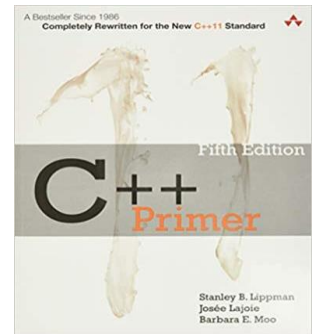
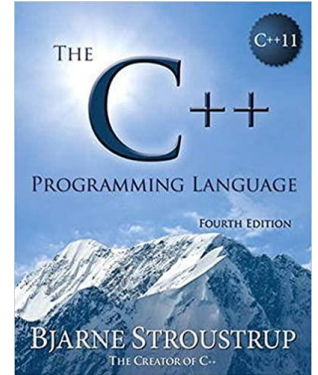
Week	Contents	Week	Contents
1 (9/3)	Introduction, C++ Basics (Ch.1)	9 (10/29)	Operator Overloading, Friends, and References (Ch.8-1)
2 (9/10)	Flow of Control (Ch.2)	10 (11/5)	Friends, and References (Ch.8-2) Strings (Ch.9),
3 (9/17)	Functions (Ch.3)	11 (11/12)	Pointers and Dynamic Arrays (Ch.10)
4 (9/24)	Parameters and Overloading (Ch.4)	12 (11/19)	Namespaces (Ch.11)
5 (10/1)	Arrays (Ch.5) Structures and Classes (Ch.6)	13 (11/26)	Inheritance (Ch.12)
6 (10/8)	No Class	14 (12/3)	Virtual Functions (Ch.13)
7 (10/15)	Constructors and Other Tools (Ch.7)	15 (12/10)	Templates (Ch.14)
8 (10/22)	Midterm Exam	16 (12/17)	Final Exam

Schedule for Lab2

Week	Contents	Week	Contents
1 (9/1)	Linux– Pt. 1	9 (10/27)	No class
2 (9/8)	Sorting Algorithm– Pt. 1	10 (11/3)	Standard Template Library – Pt. 3
3 (9/15)	Sorting Algorithm– Pt. 2	11 (11/10)	Standard Template Library – Pt. 4
4 (9/22)	Version control system (Git) – Pt. 1	12 (11/17)	Project 2
5 (9/29)	Version control system (Git) – Pt. 2	13 (11/24)	Linux– Pt. 2
6 (10/6)	Make-Up Video Lecture (No Class) Standard Template Library – Pt. 1	14 (12/1)	Problem Solving – Pt. 1
7 (10/13)	Standard Template Library – Pt. 2	15 (12/8)	Problem Solving – Pt. 2
8 (10/20)	Project 1	16 (12/15)	No class

Other References

- The C++ Programming Language
 - By Bjarne Stroustrup
- C++ Primer
 - By Stanley B. Lippman et al.
- Essential C++
 - By Stanley B. Lippman



Outline

- Intro & Basic course info
- Chapter I: C++ Basics
 - Programming paradigm
 - Comparison between C, C++ and Python
 - Simple C++ program

Before we begin...

- A Controversy



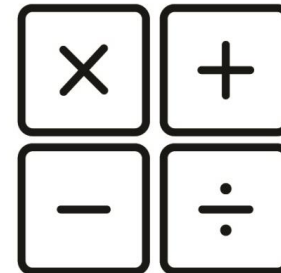
**Don't learn to code, but
study biology!**

--- Jensen Huang, Feb 2024

???

Before we begin...

- A Controversy
 - Do we need to learn arithmetic despite the calculator?
 - **Yes!**



Before we begin...

- Computational thinking & coding should be still relevant
 - The paradigm may change.
 - But, we still need to learn, know & practice the **basic**
- Useful read
 - <https://www.oreilly.com/radar/the-end-of-programming-as-we-know-it/>

Programming Paradigms

- Low-level vs. high-level programming languages
 - Machine friendly vs. human friendly
- Styles of computer programming
 - Procedural programming
 - Object-oriented programming
 - Functional programming
 - Logic programming
 - ...

Low-level vs. High-level Programming

- Low-level

- Machine code
- Assembly

<pre>1 Load r1, X 2 Load r2, Y 3 Mult r3, r2, r1 4 Load r4, A 5 Mult r2, r4, r1 6 Add r5, r2, r4 7 Mult r1, r2, r5 8 Load r3, B 9 Add r7, r1, r3 6 Registers, 9 cycles</pre>	Step 1: <pre>1 Load wire1, X 2 Load wire2, Y 3 Mult r1, wire1, wire2 4 Load wire3, A 5 Mult wire4_r2, wire1, wire3 6 Add r3, wire4, wire3</pre> Step 2: <pre>7 Mult wire5, r2, r3 8 Load wire6, B 9 Add r4, wire5, wire6</pre> 4 Registers, 2 cycles
---	---

(a) (b)

- High-level

- Basic, C, Java, Pascal, C++, Python, Perl ...

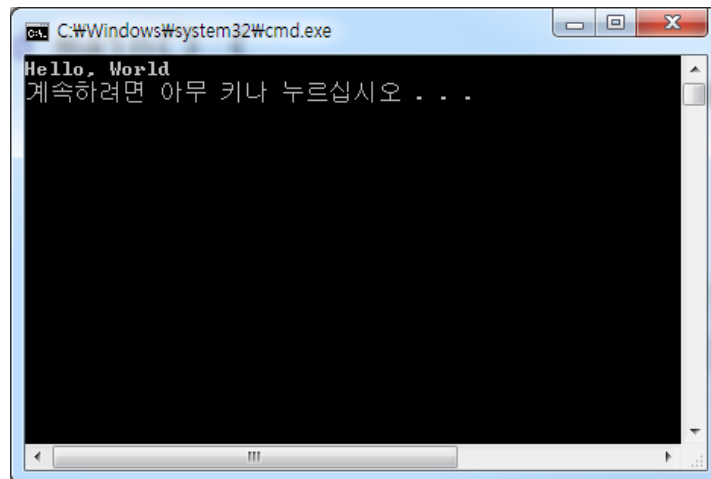
```
1 class Car:
2
3     def __init__(self, speed=0):
4         self.speed = speed
5         self.odometer = 0
6         self.time = 0
7
8     def say_state(self):
9         print(f"I'm going {self.speed}kph!")
10
11     def accelerate(self):
12         self.speed += 5
```


Comparison between C, C++ and Python

:A Simple C Program

```
#include <stdio.h>
```

```
void main() {  
    printf("Hello, world\n");  
}
```

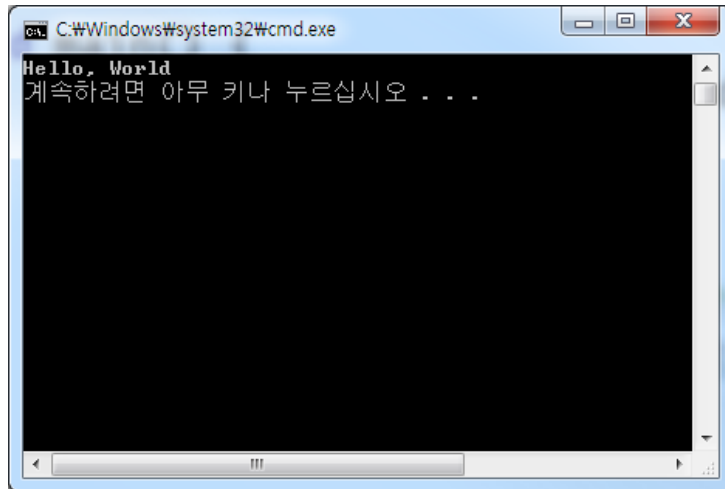


Comparison between C, C++ and Python

:A Simple C++ Program

```
#include <iostream>
```

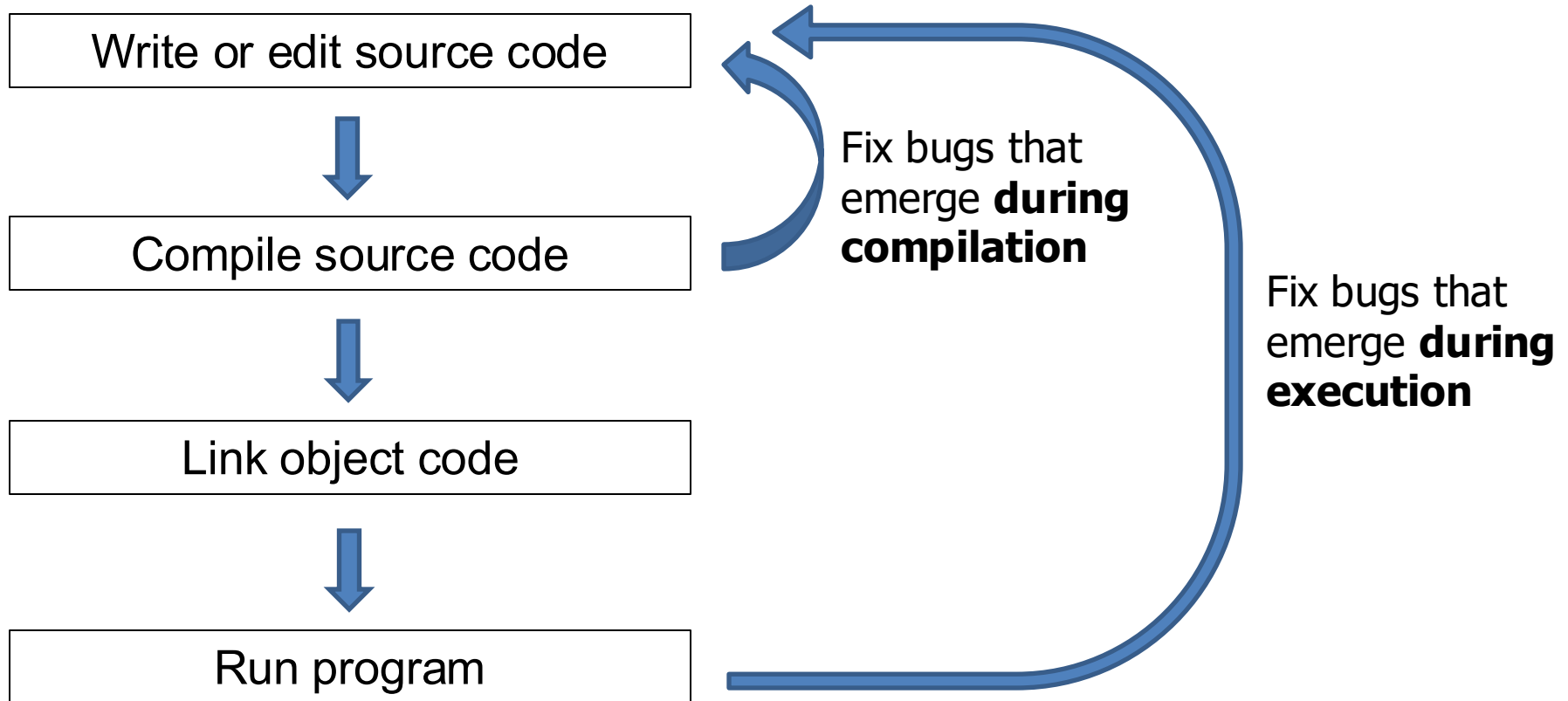
```
void main() {  
    std::cout << "Hello, world" << std::endl;  
}
```



C++ is an extension of C.

```
#include <stdio.h>  
  
void main() {  
    printf("Hello, world\n");  
}
```

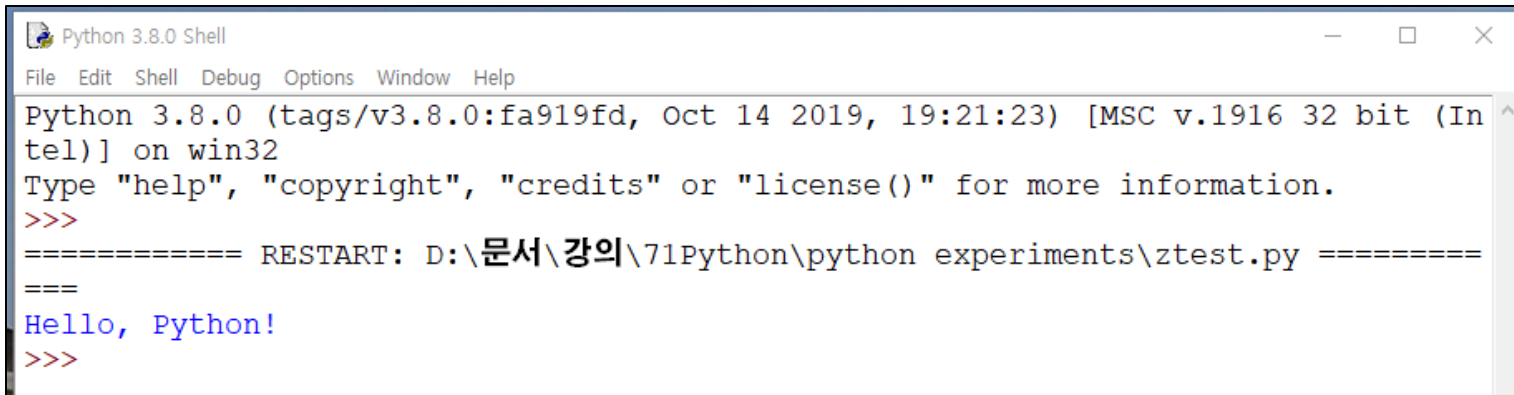
Comparison between C, C++ and Python : Compile & Execute



Comparison between C, C++ and Python

:A Simple Python Program

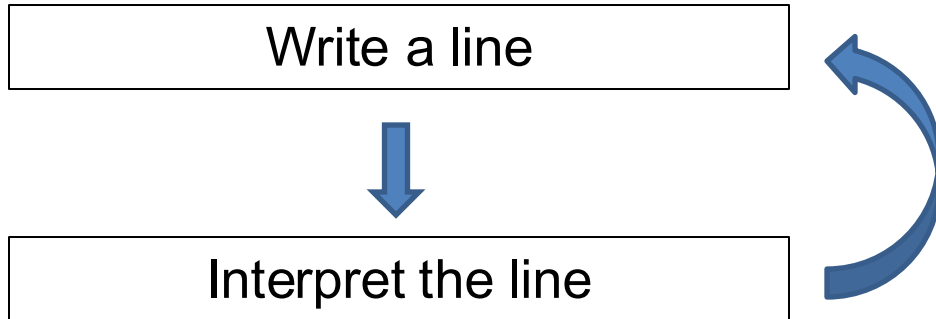
```
print("Hello, Python!")
```



```
Python 3.8.0 Shell
File Edit Shell Debug Options Window Help
Python 3.8.0 (tags/v3.8.0:fa919fd, Oct 14 2019, 19:21:23) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\문서\강의\71Python\python experiments\ztest.py =====
>>>
Hello, Python!
>>>
```

Comparison between C, C++ and Python

: In Python Shell, Interpret Each Line...



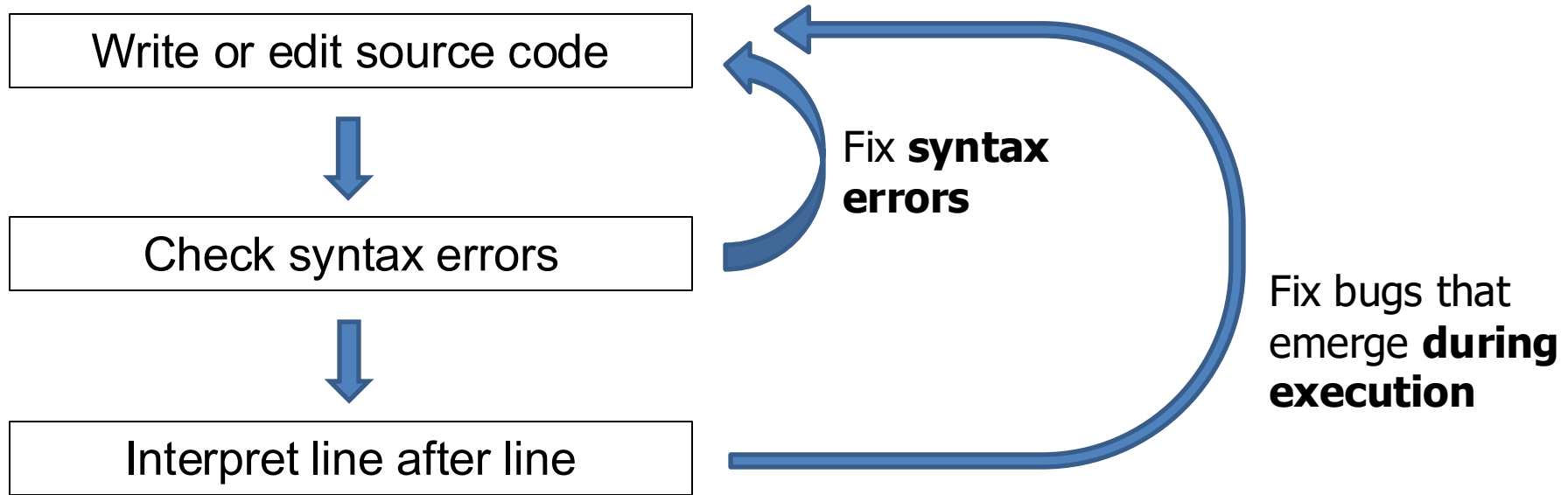
```
>>> print("Hello, Python!")
```

```
>>> def f(a):  
    b = a + 1  
    print(b)
```

In this case, the line is long...

Comparison between C, C++ and Python

: In Python IDLE, Check Syntax & Interpret

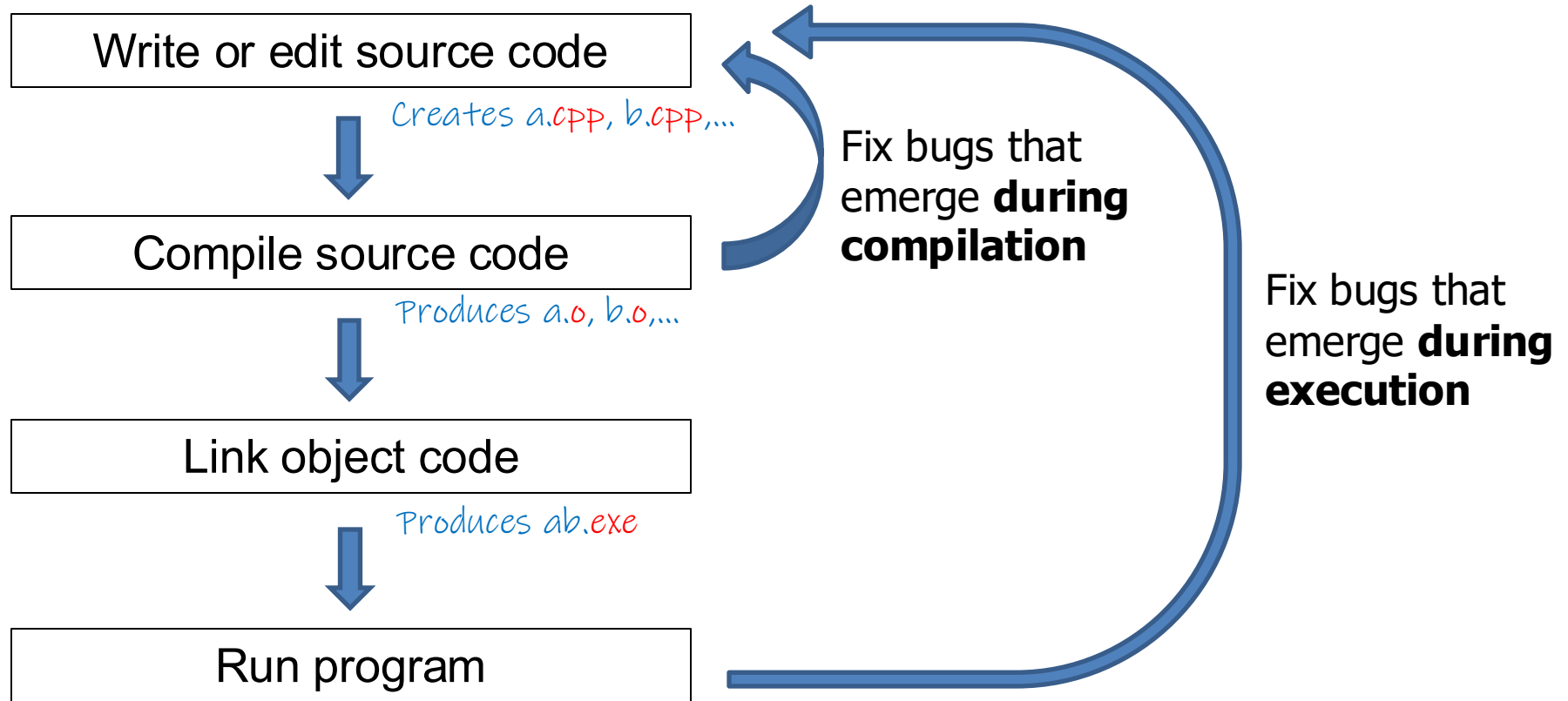


```
def f(a):  
    b += a + 1  
    print(b)  
f(3)
```

```
def f(a):  
    b = a + 1  
    print(b)  
f(3)
```

Comparison between C, C++ and Python

: In C++, Compile & Execute



From a.py, Python does not produce any extra files. Why?
It is because Python is an interpreted language.

C/C++ produces ab.exe (other than *.cpp) since it is a compiled language.

Compiler vs. Interpreter
<https://www.youtube.com/watch?v=Dx2tSsd3aFc>

Styles of Computer Programming

- Procedural programming (절차형 언어)
 - Imperative: procedures, routines, subroutines, methods, or functions.
 - E.g. Basic, C, Fortran...
- Object-oriented programming (OOP, 객체지향 언어)
 - Currently popular and powerful programming technique
 - Main characteristics
 - Encapsulation (Information hiding or abstraction)
 - Inheritance (writing reusable code)
 - Polymorphism (a single name can have multiple meanings, etc.)
 - More on these later!

Why C++?

- Improving C
 - C became popular with UNIX operating system
 - C has features of both high-level and low-level language
 - Can manipulate memory directly (low-level)
 - Easier to read and write than Assembly
 - But, it is not as easy as other high-level language
- C++ has facilities for classes
 - Supports data abstraction, OOP, and generic programming
 - Has many users and wide applications
 - Many production codes are written in C++!

Display 1.1

A Sample C++ Program (1 of 2)

Display 1.1 A Sample C++ Program

```
1  #include <iostream>
2  using namespace std;

3  int main( )
4  {
5      int numberOfLanguages;

6      cout << "Hello reader.\n"
7           << "Welcome to C++.\n";

8      cout << "How many programming languages have you used? ";
9      cin >> numberOfLanguages;

10     if (numberOfLanguages < 1)
11         cout << "Read the preface. You may prefer\n"
12              << "a more elementary book by the same author.\n";
13     else
14         cout << "Enjoy the book.\n";

15     return 0;
16 }
```

Display 1.1

A Sample C++ Program (2 of 2)

SAMPLE DIALOGUE 1

Hello reader.

Welcome to C++.

How many programming languages have you used? 0 ← *User types in 0 on the keyboard.*

Read the preface. You may prefer

a more elementary book by the same author.

SAMPLE DIALOGUE 2

Hello reader.

Welcome to C++.

How many programming languages have you used? 1 ← *User types in 1 on the keyboard.*

Enjoy the book

Variables

- Identifiers

```
x x1 x_1 _abc ABC123z7 sum RATE count data2 bigBonus
```

- Keywords/reserved words

- **Case-sensitive**

```
rate RATE Rate
```

- Illegal usages

```
12 3X %change data-1 myfirst.c PROG.CPP
```

- Use meaningful names!

- Variables

- **A memory location** to store data for a program

- Must **declare** all data before use in program

```
int count, numberOfDragons, numberOfTrolls;  
double distance;
```

Data Types:

Display 1.2 Simple Types (1 of 2)

Display 1.2 Simple Types

TYPE NAME	MEMORY USED	SIZE RANGE	PRECISION
<code>short</code> (also called <code>short int</code>)	2 bytes	–32,768 to 32,767	Not applicable
<code>int</code>	4 bytes	–2,147,483,648 to 2,147,483,647	Not applicable
<code>long</code> (also called <code>long int</code>)	4 bytes	–2,147,483,648 to 2,147,483,647	Not applicable
<code>float</code>	4 bytes	approximately 10^{-38} to 10^{38}	7 digits
<code>double</code>	8 bytes	approximately 10^{-308} to 10^{308}	15 digits

Depends on the OS
and targeted
architectures

Data Types:

Display 1.2 Simple Types (2 of 2)

<code>long double</code>	10 bytes	approximately 10^{-4932} to 10^{4932}	19 digits
<code>char</code>	1 byte	All ASCII characters (Can also be used as an integer type, although we do not recommend doing so.)	Not applicable
<code>bool</code>	1 byte	<code>true</code> , <code>false</code>	Not applicable

Assignment Statements

- Initializing data in declaration statement
 - Results "undefined" if you don't initialize
 - `int myValue = 0;`
- Assigning data during execution
 - Lvalues (left-side) & Rvalues (right-side)
 - **Lvalues** must be variables
 - **Rvalues** can be any expression
 - Example:
`distance = rate * time;`
Lvalue: "distance"
Rvalue: "rate * time"

Assignment Statements : Shorthand Notations

EXAMPLE	EQUIVALENT TO
<code>count += 2;</code>	<code>count = count + 2;</code>
<code>total -= discount;</code>	<code>total = total - discount;</code>
<code>bonus *= 2;</code>	<code>bonus = bonus * 2;</code>
<code>time /= rushFactor;</code>	<code>time = time / rushFactor;</code>
<code>change %= 100;</code>	<code>change = change % 100;</code>
<code>amount *= cnt1 + cnt2;</code>	<code>amount = amount * (cnt1 + cnt2);</code>

Assignment Rules / Compatibility

- Data assignments should be compatible!

```
int intVariable;  
intVariable = 2.99;
```

Type mismatch!

- Generally, cannot place value of one type into a variable of another type!

- Only integer part "fits", so that's all that goes into the intVariable
- Called "implicit" or "automatic type conversion"

- Some special cases are allowed

```
double doubleVariable;  
doubleVariable = 2;
```

- Both legal and acceptable

- Literals: considered “constants” (cannot be changed)

- E.g.) 2, 5.75, 'Z', "Hello World"

Escape Sequences

- "Extend" the character set
- Backslash, \ preceding a character
 - Instructs compiler: a special "escape character" is coming
 - Following character treated as "escape sequence char"
 - See Display 1.3 in the next slide

Display I.3

Some Escape Sequences (I of 2)

SEQUENCE	MEANING
<code>\n</code>	New line
<code>\r</code>	Carriage return (Positions the cursor at the start of the current line. You are not likely to use this very much.)
<code>\t</code>	(Horizontal) Tab (Advances the cursor to the next tab stop.)
<code>\a</code>	Alert (Sounds the alert noise, typically a bell.)
<code>\\</code>	Backslash (Allows you to place a backslash in a quoted expression.)
<code>\'</code>	Single quote (Mostly used to place a single quote inside single quotes.)

Display 1.3

Some Escape Sequences (2 of 2)

<code>\"</code>	Double quote (Mostly used to place a double quote inside a quoted string.)
-----------------	--

The following are not as commonly used, but we include them for completeness:

<code>\v</code>	Vertical tab
-----------------	--------------

<code>\b</code>	Backspace
-----------------	-----------

<code>\f</code>	Form feed
-----------------	-----------

<code>\?</code>	Question mark
-----------------	---------------

Arithmetic Precision

- Precision of calculations
 - VERY important consideration!
 - Expressions in C++ might not evaluate as you'd "expect"!
 - "Highest-order operand" determines type of arithmetic "precision" performed

Arithmetic Precision : Examples

- Examples:
 - `17/5` evaluates to 3 in C++
 - Both operands are integers
 - Integer division is performed (note the number *is NOT rounded.*)
 - `17.0/5` equals 3.4 in C++
 - Highest-order operand is "double type"
 - Double "precision" division is performed
 - `int intVar1 =1, intVar2=2;
intVar1/intVar2;`
 - Performs integer division
 - Result: 0

Arithmetic Precision : Examples

- Examples:

- When `feet=15000` is an `int` type,

```
totalPrice = 5000 * (feet/5280.0);
```

evaluates to $5000 * 2.84 = 14200$.

- But,

```
totalPrice = 5000 * (feet/5280);
```

will become $5000 * 2 = 10000$, which is a serious underestimate!

Individual Arithmetic Precision

- Calculations are done "**one-by-one**"
 - $1/2/3.0/4$ performs 3 separate divisions.
 - $1/2$ equals 0
 - $0/3.0$ equals 0.0
 - $0.0/4$ equals 0.0
- So not necessarily sufficient to change just "one operand" in a large expression
 - Must keep in mind all individual calculations that will be performed during evaluation

Type Casting (Type Conversion)

- Two types

- **Implicit** — also called as "**Automatic**"

- Done FOR you automatically

e.g., `17/5.5` causes an "implicit type cast" to take place, casting the `17` to `17.0`

- **Explicit**

- Programmer specifies conversion with cast operator

e.g., `(double)17/5.5` is the same as above (explicit cast)
`(double)myInt/myDouble`

- A newer convention: `static_cast<double>(m)`

Shorthand Increment/Decrement Operators

- Increment & Decrement Operators

- Increment operator, **++**

`intVar++;` is equivalent to
`intVar = intVar + 1;`

- Decrement operator, **--**

`intVar--;` is equivalent to
`intVar = intVar - 1;`

Shorthand Operators: Two Options

- **Post-Increment**

`intVar++`

- Uses current value of variable, THEN increments it

- **Pre-Increment**

`++intVar`

- Increments variable first, THEN uses new value

Post-Increment in Action

- Post-Increment in expressions:

```
int n = 2;  
int valueProduced = 2*(n++);  
cout << valueProduced << "\n";  
cout << n << "\n";
```

- This code segment produces the output:

```
4  
3
```

Pre-Increment in Action

- Now using Pre-increment:

```
int n = 2;  
int valueProduced = 2*(++n);  
cout << valueProduced << "\n";  
cout << n << "\n";
```

- This code segment produces the output:

6
3

Console Input/Output

- Simple console I/O **objects**: `cin`, `cout`, `cerr`
 - Defined in the C++ library called `<iostream>`
- Must have the following lines (called pre-processor directives) near start of file:

```
#include <iostream>
using namespace std;
```

- Tells C++ to use appropriate library so we can use the I/O objects `cin`, `cout`, `cerr`

Separating Lines of Output

- New lines in output
 - `'\n'` is escape sequence for the char "newline"
- A second method: **object** `endl`

- Examples:

```
cout << "Hello World\n";
```

- Sends string "Hello World" to display, & escape sequence `'\n'`, skipping to next line

```
cout << "Hello World" << endl;
```

- Same result as above

Console Output

- What can be outputted?
 - Any data can be outputted to display screen
 - Variables
 - Constants
 - Literals
 - Expressions (which can include all of above)
 - E.g.,

```
cout << numberOfGames << " games played.";
```

 - 2 values are outputted
 - "value" of the variable `numberOfGames`
 - literal string `" games played."`
 - Cascading: multiple values in one `cout` is allowed
 - Equivalent to

```
cout << numberOfGames;  
cout << " games played.";
```


Input using `cin`

- `cin` for input, `cout` for output
- Differences:
 - "`>>`" (extraction operator) points opposite
 - Think of it as "pointing toward where the data goes"
 - Object name "`cin`" used instead of "`cout`"
 - No literals allowed for `cin`
 - Must input "to a variable"
- `cin >> num;`
 - Waits on-screen for keyboard entry
 - Value entered at keyboard is "assigned" to `num`

Formatting numbers with decimal point

- Consider `double price=78.5`

```
cout << "The price is $" << price << endl;
```

- Possible outputs:

```
The price is $78.500000  
The price is $78.5  
The price is $7.850000e01
```

- Ideally, we would want

```
The price is $78.50
```

what we can do:

```
cout.setf(ios::fixed);  
cout.setf(ios::showpoint);  
cout.precision(2);
```

Member function of cout!

Program Style: Comments

- “Just enough” comments are needed
 - Whenever something is important or not obvious
 - Too many comments are as bad as too few
- Two methods for comments
 - // Two slashes indicate entire line is to be ignored
 - /* Delimiters indicates everything between is ignored */

```
/*This is a comment that spans  
three lines. Note that there is no comment  
symbol of any kind on the second line.*/
```

```
distance = speed * time; //Computes the distance traveled.
```

Program Style: Identifier naming

- Identifier naming
 - Must begin with an alphabet character (a-z,A-Z) or underscore (`_`)
 - Usually use `ALL_CAPS` for constants
 - Most important: Use `MEANINGFUL NAMES!`

Libraries

- C++ Standard Libraries

```
#include <Library_Name>
```

```
#include <iostream>
```

- Directive to "add" contents of library file to your program
- Called "preprocessor directive"
 - Executes before compiler, and simply "copies " library file into your program file

- C++ has many libraries

- Input/output, math, strings, etc.

Namespaces

- Collection of name definitions
 - For now, we are only interested in the namespace `std`
 - Has all standard library definitions we need
 - One name can be given different definitions in two namespaces
- A simple program that uses console I/O would begin

```
#include <iostream>
using namespace std;
```

When you want to use the
entire standard library

```
#include <iostream>
using std::cin;
using std::cout;
using std::endl;
```

If only want to use
`cin`, `cout`, `endl`

Summary I

- C++ is a compilation language
- Variables, Expressions, Assignments
 - Case-sensitive
 - Use meaningful names for variables and constants
 - Variables must be declared before use (initialize!)
 - Use care in numeric manipulation
 - Precision, parentheses, order of operations

Thank you!

Web: <http://mindlab-snu.github.io>

E-mail: tsmoon@snu.ac.kr